

Estrategia de optimización del LTV mediante predicción de churn en el sector telco

ETAPA 1: COMPRENSIÓN DE NEGOCIO.

El dataset seleccionado trata sobre el fenómeno conocido como Churn que es la tasa de abandono de los clientes que consumen tus servicios por ejemplo, esto tiene un impacto directo en la rentabilidad de tu empresa.

En mi trabajo actual, estudio y analizo datos relacionados con el rendimiento de campañas publicitarias, por lo que es importante que es para una empresa el churn de los clientes, el LTV, el potencial de un cliente no es captarlo sino captarlo y retenerlo el máximo tiempo posible, cuantos más clientes tengas en tu customer base más clientes podrás captar en el futuro.

Gracias a la IA las empresas pueden preparar diferentes escenarios de cara a no solventar este problema pero si hacer todo lo posible para que su afectación sea lo más mínima posible.

El motor de toda empresa son sus clientes, por ello predecir su comportamiento es algo básico para preservar el crecimiento de la empresa.

1.1. OBJETIVOS DE NEGOCIO

Predicción preventiva → Utilizar la IA para anticiparnos al comportamiento del cliente, para ello detectaremos patrones de comportamientos de abandono.

Escenarios de actuación → Preparar diferentes escenarios estratégicos para que la empresa pueda reaccionar con campañas de fidelización.

Preservación del crecimiento → Asegurar la estabilidad de la customer base para garantizar estabilidad económica a largo plazo y escalabilidad del negocio.

1.2. BENEFICIOS ESPERADOS

Minimización del impacto → El abandono no se puede eliminar por completo, pero la IA permite que la tasa sea más baja de lo habitual.

Optimización del marketing → Podremos centrar esfuerzos únicamente a estos clientes que respondan a estos patrones estudiados, optimizando recursos y mejorando el rendimiento de las campañas.

Mejora de la rentabilidad → Al anticiparnos al abandono de los clientes, tendremos una customer base más grande y probablemente en este apartado la empresa sea rentable, evidentemente la rentabilidad de una empresa no solo de sus clientes.

ETAPA 2: ENTENDIMIENTO DE LOS DATOS EDA

2.1 Carga de datos y estructura inicial

En este primer paso identificamos que tenemos un total de 7043 registros y 21 columnas.

Vemos que TotalCharges aparece en formato texto en lugar de número, hallazgo importantísimo que solventaremos en el siguiente apartado de código.

```
print(df.info())
print(df.describe())
```

```
*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   customerID            7043 non-null  object  
1   gender                7043 non-null  object  
2   SeniorCitizen          7043 non-null  int64   
3   Partner               7043 non-null  object  
4   Dependents            7043 non-null  object  
5   tenure                7043 non-null  int64   
6   PhoneService          7043 non-null  object  
7   MultipleLines          7043 non-null  object  
8   InternetService       7043 non-null  object  
9   OnlineSecurity        7043 non-null  object  
10  OnlineBackup          7043 non-null  object  
11  DeviceProtection      7043 non-null  object  
12  TechSupport           7043 non-null  object  
13  StreamingTV           7043 non-null  object  
14  StreamingMovies       7043 non-null  object  
15  Contract              7043 non-null  object  
16  PaperlessBilling      7043 non-null  object  
17  PaymentMethod         7043 non-null  object  
18  MonthlyCharges         7043 non-null  float64  
19  TotalCharges          7043 non-null  object  
20  Churn                 7043 non-null  object  
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
None
```

	SeniorCitizen	tenure	MonthlyCharges
count	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692
std	0.368612	24.559481	30.090047
min	0.000000	0.000000	18.250000
25%	0.000000	9.000000	35.500000
50%	0.000000	29.000000	70.350000
75%	0.000000	55.000000	89.850000
max	1.000000	72.000000	118.750000

2.2 Limpieza técnica necesaria para el análisis

Con estas líneas de código:

```
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
print(df.isnull().sum())
```

Convertimos el campo a numérico y además con la segunda línea aprovechamos para ver si existen valores null, básico para limitar el ruido a la hora de entrenar el algoritmo, nuestro objetivo ahora es mejorar la calidad de nuestros datos de cara al entrenamiento de los mismos y que por tanto la fase de despliegue salga perfecta.

Con la ejecución de nuestro código vemos que tenemos 11 valores null. Es básico el perfecto conocimiento del dataset y de la lógica de negocio, podríamos pensar que carecemos de estos datos, pero no, estos valores null valen lo mismo que si tuviéramos un número, puesto que se refiere a que son clientes nuevos, con menos de 1 mes de antigüedad.

```
** customerID      0
   gender          0
   SeniorCitizen   0
   Partner         0
   Dependents      0
   tenure          0
   PhoneService    0
   MultipleLines   0
   InternetService 0
   OnlineSecurity  0
   OnlineBackup    0
   DeviceProtection 0
   TechSupport     0
   StreamingTV     0
   StreamingMovies 0
   Contract        0
   PaperlessBilling 0
   PaymentMethod   0
   MonthlyCharges  0
   TotalCharges    11
   Churn           0
   dtype: int64
```

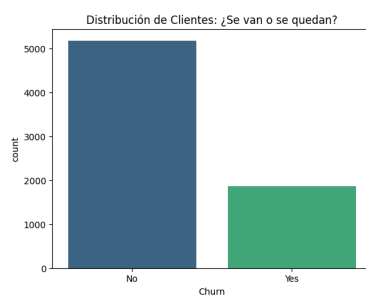
2.3 Análisis de la variable objetivo → CHURN

```
print(df['Churn'].value_counts(normalize=True))
```

La salida de este código nos dice que

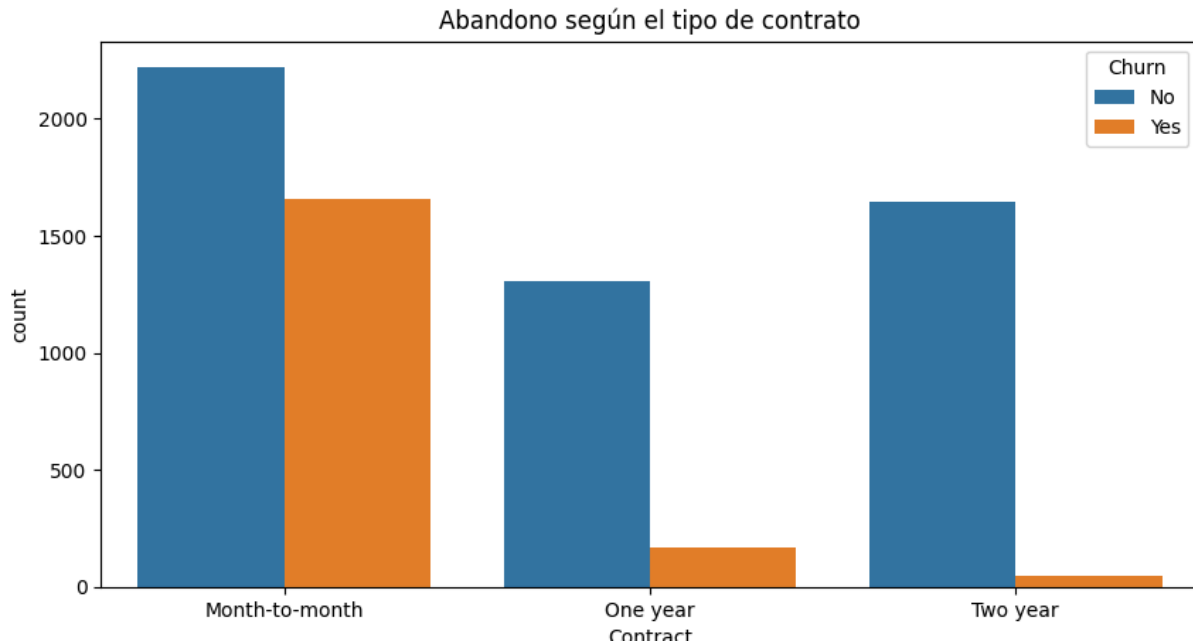
```
Churn
No      0.73463
Yes     0.26537
```

El 26.5% de los clientes dejaron de disfrutar de nuestros servicios. Este es nuestro punto de partida, el modelo intentará predecir ese 26.5% de casos positivos.



2.4 Análisis de relaciones clave

Los clientes no abandonan el consumo por arte de magia, hay variables del entorno que les afectan y les motivan a abandonar.



Podemos llegar a una conclusión al ver estos datos y es que los contratos con una estabilidad menor concentran la mayor tasa de abandono. Esto confirma que la estabilidad del contrato es una métrica crítica para el objetivo principal de la empresa que es retener usuarios y que por tanto el LTV de cada cliente sea mayor.

CONCLUSIONES ANÁLISIS DE NEGOCIO

A partir de los datos obtenidos, podemos extraer las siguiente conclusiones.

Estabilidad y retención: Existe una relación directa entre la duración del contrato y la fidelidad de los clientes. Los contratos con menor estabilidad (month to month) concentran la mayor tasa de abandono, mientras que cuanto mayor sea la estabilidad del mismo la tasa de abandono cae drásticamente. Esto confirma que la estabilidad del contrato es la métrica más importante y crítica de cara a aumentar el LTV por cliente.

Fase de evaluación: Los datos revelan un mayor número de clientes durante la modalidad de contrato mes a mes, debido a que es la fase donde el cliente evalúa el servicio/producto.

En este caso también podemos llegar a una hipotética conclusión, y es que el mayor volumen de clientes tanto que deciden irse como quedarse es el del month to month, la fase en la que el cliente prueba el producto, el cliente evalúa si quiere permanecer allí o realmente no le interesa el producto/servicio, por ello es la fase más importante y crítica para una empresa, aquí y gracias a estos modelos de machine learning es donde el

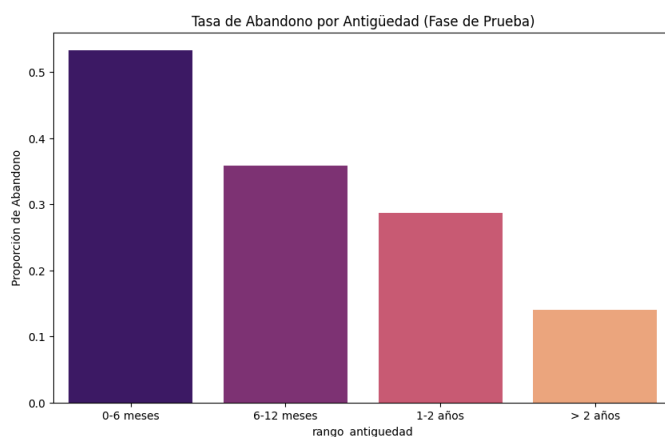
empresario debe dividir y optimizar recursos económicos, trata de que el cliente pase de un contrato mes a mes a yearly por ejemplo.

Este código es pura ingeniería de características pero lo he visto muy interesante de cara a sacar conclusiones de negocio.

He creado una nueva variable que divida o agrupe los clientes por segmentos lógicos de negocio, es decir, por grupos de antigüedad, para de esta manera visualizar la tasa de abandono en cada segmento

```
df['rango_antiguedad'] = pd.cut(df['tenure'], bins=[0, 6, 12, 24, 72],
                                labels=['0-6 meses', '6-12 meses', '1-2 años', '> 2 años'])

plt.figure(figsize=(10, 6))
sns.barplot(x='rango_antiguedad', y=df['Churn'].map({'Yes': 1, 'No': 0}), data=df, ci=None, palette='magma')
plt.title('Tasa de Abandono por Antigüedad (Fase de Prueba)')
plt.ylabel('Proporción de Abandono')
plt.show()
```



Con este gráfico vemos claramente como la tasa de abandono se encuentra en los primeros meses donde el tipo de contrato es mensual o bimensual.

Decisión estratégica Gracias a este modelo de machine learning, la empresa puede identificar a estos clientes en su fase más vulnerable y optimizar recursos económicos, dirigiendo campañas específicas para incentivar el paso de un contrato mensual a uno anual antes de que se cumpla el primer semestre que como hemos visto agrupa las mayores probabilidades de abandono.

Evidentemente, la duración del contrato no es la única variable que influye en el churn de los clientes, pero es interesante ver como con machine learning se puede hacer un estudio mucho más profundo de lo que sucede en el consumidor y lo más importante fundamentado en datos.

Una vez hemos comprendido e identificado los patrones de comportamiento clave y las inconsistencias en datos como los valores null o valores en un formato erróneo, podemos pasar a la etapa 3, la de preparación de los datos. En esta fase será donde se realice la ingeniería de características que permita al algoritmo aprender de manera eficiente y sin sesgos en los datos.

ETAPA 3: PREPARACIÓN DE LOS DATOS

3.1 Ingeniería de características y selección

No todas las columnas sirven para predecir por lo que lo primero que haremos será eliminar el customer ID.

```
df_preparacion = df.drop('customerID', axis=1)
```

Además también convertiremos las categorías de texto yes/no a un formato booleano para que el algoritmo pueda procesarlos de manera correcta. Para hacer esto usaremos un mapeo mediante un diccionario, su resultado final es parecido a aplicar una función lambda.

```
df_preparacion['Churn'] = df_preparacion['Churn'].map({'Yes': 1, 'No': 0})
```

3.2 División y escalado

Lo que conseguiremos con las siguientes líneas de código es que variables con números muy grandes no dominen de manera injusta sobre las variables más pequeñas, de esta manera no habrán sesgos en el algoritmo. Aplicaremos un escalado estándar a las variables continuas como los cargos mensuales y la antigüedad, asegurando que todos los campos tengan el mismo peso jerárquico ante el algoritmo y como he dicho el resultado no pinte datos sesgados.

Esta normalización la haremos mediante la librería: `from sklearn.preprocessing import StandardScaler`.

Además vamos a dividir el dataset en dos conjuntos, uno de entrenamiento para que el modelo aprenda los patrones de la tasa de abandono y otro de prueba para poder evaluar que tan bien predice el modelo con datos que no ha visto. Esto lo haremos mediante `from sklearn.model_selection import train_test_split`.

Definimos las variables de predicción y el objetivo: Para el entrenamiento del modelo se separa el conjunto de datos en dos estructuras $\rightarrow$ x e y.

- **Variable independiente (x):** Encontramos el conjunto de atributos que definen al cliente como su antigüedad, servicios, cargos, etc)
- **Variable dependiente (y):** La tasa de abandono, es decir, la variable churn, que es lo que el modelo debe de aprender a identificar.

El código que usaré es el siguiente:

```
x = df_preparacion.drop('Churn', axis=1)  
y = df_preparacion['Churn']
```

La primera línea, es decir, la variable independiente crea la matriz de características.

Mientras que en la segunda línea se crea el vector de la variable objetivo.

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

Ahora dividimos el dataset, 80% para entrenar y 20% para evaluar.

Con esto hemos acabado las etapas de preparación de los datos, ahora es el turno de comenzar con las etapas de “inteligencia”.

ETAPA 4: MODELADO

4.1 Selección y diseño de algoritmos

En este caso de uso he decidido probar dos arquitecturas distintas, el objetivo principal es detectar cuales de estos dos algoritmos identifica mejor el patrón de comportamiento de los clientes a la hora de abandonar que se ha detectado anteriormente en la fase exploratoria de los datos.

Para ello me he decantado por los siguientes 2 modelos.

4.1.1 Regresión logística

Es el estándar de problemas de clasificación binaria (en este caso tenemos muchas variables booleanas). Funciona estimando probabilidades.

Durante el análisis del comportamiento de fuga, es evidente que no todas las variables influyen de la misma manera en el cliente, como hemos visto en la fase de EDA la métrica de estabilidad del contrato ha sido bastante crítica. Esta arquitectura es capaz de asignar “pesos” permitiendo que el algoritmo sea consciente de que factores tienen un impacto real en la tasa de abandono.

Esto representa una ventaja competitiva a la hora de presentarlo al departamento de negocio de la empresa. Estos únicamente tienen como objetivo optimizar recursos y maximizar resultados, este modelo permite explicar de forma clara en donde la empresa tiene que enfocarse e invertir recursos de manera directa para poder reducir la brecha y mejorar la escalabilidad de la empresa a medio largo plazo.

4.1.2 Random forest

Es una arquitectura de aprendizaje supervisado que utiliza árboles de decisión.

Al igual que con el modelo anterior se decía que se podían dar pesos diferentes a cada variable para que cada una tuviera una importancia determinada de cara a detectar la fuga de clientes, este modelo permite identificar que la causa de abandono no solo está ligada a una variable sino la combinación específica de factores, yendo mucho más allá que simplemente limitarse a decir, la causa de fuga de clientes es dada por la baja estabilidad del contrato. Por ejemplo, puede detectar que un cliente senior con fibra óptica y pago por

cheque electrónico tiene una probabilidad de fuga radicalmente distinta a un perfil joven con los mismos servicios.

El random forest al promediar múltiples árboles de decisión reduce significativamente el riesgo de sesgos individuales, ofreciendo una predicción más estable y fiable a la hora de toma de decisiones

Otra ventaja clara de este modelo es que permite saber que es lo que está generando una mayor tasa de churn, esto es inteligencia de mercado, permitiendo por tanto desarrollar estrategias de negocio fundamentadas en datos de cara invertir en las áreas que mayor impacto tienen en el LTV del cliente. Como se dijo en la primera parte de este trabajo en la fase 1 la relacionada con entendimiento de negocio, uno de los objetivos principales era:

“Preservación del crecimiento → Asegurar la estabilidad de la customer base para garantizar estabilidad económica a largo plazo y escalabilidad del negocio.
”

Con esto estamos enfocándonos directamente aquí, reducir al máximo la fuga con datos, para aumentar por tanto la customer base generando argumentos de estabilidad financiera y escalabilidad empresarial a medio largo plazo.

Vamos a ver todo esto en código

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

modelo_logistico = LogisticRegression(max_iter=1000, random_state=42)

modelo_rf = RandomForestClassifier(n_estimators=100, random_state=42)
```

Importamos librerías para el modelo logístico y random forest.

En el caso del modelo logístico lo instanciamos con un límite de iteraciones alto para asegurar la convergencia.

Por otro lado el random forest lo desplegamos con 100 árboles de decisión.

Entrenamos ambos modelos.


```
print("--- El modelo de regresión logística esta entrenando.... ---")

combined_train = pd.concat([X_train, y_train], axis=1)
combined_train_cleaned = combined_train.dropna()

X_train_cleaned = combined_train_cleaned.drop(columns=y_train.name)
y_train_cleaned = combined_train_cleaned[y_train.name]

modelo_logistico.fit(X_train_cleaned, y_train_cleaned)

--- El modelo de regresión logística esta entrenando.... ---
LogisticRegression
LogisticRegression(max_iter=1000, random_state=42)
```

Este es más largo, algo falló y Gemini me propuso desde el mismo IDE esta solución.

Para el caso de random forest logré entrenarlo únicamente con una línea de código, al igual que estaba intentando para el caso anterior.

```
print("Entrenando modelo de Random Forest...")
modelo_rf.fit(X_train, y_train)

... Entrenando modelo de Random Forest...
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

Una vez hemos limpiado y preparado los datos para poder entrenar ambos algoritmos pasamos a la quinta etapa del método CRISP-DM, esta etapa es la de evaluación.

ETAPA 5: EVALUACIÓN

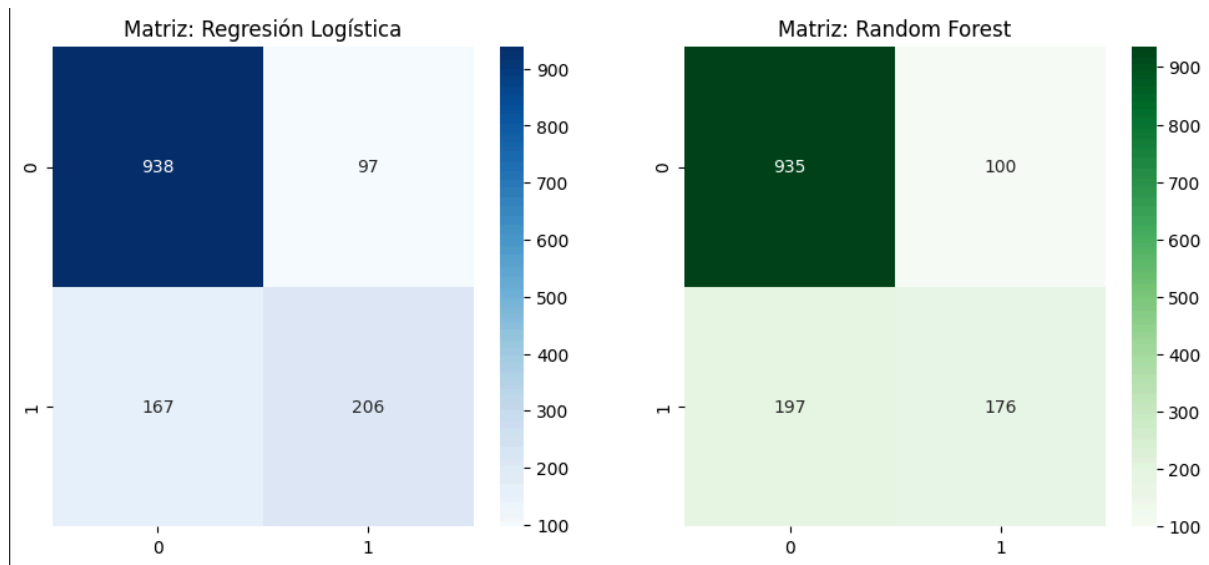
Esta etapa es una etapa crítica por que es donde veremos si el modelo realmente sirve para los objetivos de negocio que se plantearon al principio, concretamente en la etapa 1.

5.1 Comparativa de modelos (Métricas globales)

Tras enfrentar ambos modelos a datos no vistos durante el entrenamiento, se han obtenido los siguientes indicadores de rendimiento:

- **Exactitud:** El modelo de regresión logística (0.81) supera ligeramente al de random forest (0.79). Esto confirma que, para la estructura actual de datos, una aproximación lineal es más precisa.
- **F1-Score:** Con un valor de 0.61, la regresión logística demuestra ser mas equilibrada para identificar clientes en riesgo de abandono en comparación con el modelo de ensamble.

5.2 Análisis de la matriz de confusión



Al analizar los mapas de calor, observamos el comportamiento real del algoritmo frente a la toma de decisiones:

- **Efectividad en lealtad:** De los 1.035 clientes que no se iban, la regresión logística acertó en 938 casos (91%). El modelo es excelente filtrando por clientes seguros.
- **Desafío del churn:** Se identificaron correctamente 206 clientes que efectivamente se fueron pero sin embargo existen 167 falsos negativos. Este es el punto clave para mejorar en futuras iteraciones.

5.3 Hallazgos del informe de clasificación

El análisis detallado del informe revela una dicotomía importante para el negocio:

- **Precisión(0.68):** Esto significa que cuando el modelo marca a un cliente como riesgo de fuga tiene un 68% de posibilidades de tener razón. Gracias a esto la empresa evita malgastar recursos de marketing y financieros en aquellos clientes que el algoritmo no reconoce en riesgo de fuga. Para este modelo considero que hay mucho margen de mejora y no está para ponerse en producción, necesita mas trabajo y fiabilidad para poder venderlo dentro del departamento de negocio.
- **Recall(0.55):** Actualmente detectamos al 55% de los clientes que deciden dejar de disfrutar de nuestros servicios. Sigo considerando que tiene una probabilidad baja para tener fe ciega en él, puede servirnos como una buena herramienta de apoyo, pero no deberíamos de fiarnos al 100% en ella, le queda tiempo de desarrollo.

5.4 Conclusiones y selección de modelo

- **Modelo ganador:** Se selecciona la regresión logística. No solo tiene mejor Accuracy sino que su recall es superior al de random forest. Detecta a 30 clientes en riesgo de fuga más, aunque repito que hay que seguir optimizando más y más el algoritmo.

- **Impacto en LTV:** Al identificar correctamente a más de la mitad de los clientes que deciden marcharse, es una gran ayuda para la empresa pues detecta a un 55% de clientes que van a irse de la empresa, aunque sigamos sin saber detectar a un 45% si lo hacemos para la otra mitad, por tanto podemos empezar a focalizar esfuerzos y campañas de fidelización mucho más específicas y por tanto seguramente más fructíferas porque muy probablemente ese % de clientes que está detectando como riesgo de fuga comparte características, podemos tratarlos como un segmento diferente del resto de nuestra customer base y como he dicho dirigir esfuerzos y campañas de fidelización específicas a ellos.
- **Decisión estratégica:** A pesar de los errores, contar con este modelo permite pasar de una estrategia reactiva a una preventiva, fundamentada en datos y no en suposiciones.

ETAPA 6: DESPLIEGUE

6.1 Exportación y persistencia del modelo.

Para garantizar que la solución sea escalable y utilizable, he procedido a la serialización del modelo de regresión logística, modelo ganador, mediante la librería joblib. Permite guardar el conocimiento extraído de los datos en un archivo ligero .pkl, de esta manera evitamos reproducir de manera repetitiva el proceso de entrenamiento en el futuro.

```
import joblib

joblib.dump(modelo_logistico, 'modelo_fuga_clientes_final.pkl')

['modelo_fuga_clientes_final.pkl']

▶ joblib.dump(scaler, 'escalador_telco.pkl')

print("--- DESPLIEGUE FINALIZADO ---")
print("Archivos generados: 'modelo_fuga_clientes_final.pkl' y 'escalador_telco.pkl'")

... --- DESPLIEGUE FINALIZADO ---
Archivos generados: 'modelo_fuga_clientes_final.pkl' y 'escalador_telco.pkl'
```

6.2 Integración en el flujo de negocio.

El despliegue de este modelo permite a la empresa automatizar la detección de la siguiente manera:

- **Input de datos:** Cada sistema carga datos de los clientes actuales.
- **Procesamiento:** El archivo de escalador emite una probabilidad de fuga.
- **Acción:** Los clientes con una probabilidad superior al 50% son enviados automáticamente al equipo de fidelización.

6.3 Beneficios del despliegue estratégico

- **Reducción del churn:** Al actuar de forma preventiva sobre los 206 casos positivos que el modelo detectó en la evaluación la empresa puede reducir la tasa de abandono de forma inmediata.
- **Optimización de recursos:** El departamento de marketing ya no lanzará campañas masivas, sino que se centrará en el segmento de alto riesgo, maximizando el LTV y el retorno de la inversión publicitaria.

CONCLUSIÓN GENERAL DEL PROYECTO

El presente proyecto, desarrollado bajo la metodología CRISP-DM, ha permitido transformar un conjunto de datos brutos de una base de datos en una herramienta estratégica de toma de decisiones. A lo largo de las 6 etapas se han alcanzado los siguientes hitos:

- **Conexión estratégica:** Se identificó que el churn de los clientes no es un evento aleatorio, si no un comportamiento de patrones predecibles, en este caso, especialmente relacionados y vinculados con la estabilidad de los contratos y la antigüedad del cliente en la empresa
- **Validación técnica:** Mediante un análisis comparativo, se demostró que la regresión logística ofrece un equilibrio óptimo para este negocio, logrando una exactitud del 81%.
- **Valor económico:** El modelo no solo predice, permite a la empresa optimizar sus recursos. Al identificar correctamente a los clientes en riesgo con una precisión de 68%, se garantiza que las inversiones de fidelización se dirijan a los perfiles correctos, maximizando el LTV y preservando la customer base.
- **Escalabilidad:** Con el despliegue final, la solución queda lista para integrarse en un entorno de producción, aunque como analista de datos de marketing digital que soy, no daría en estos momentos aún el visto bueno a desplegarlo como versión final, soy consciente de que hay que seguir afinando en modelo para que el equipo de marketing pueda usarlo como una buena herramienta para apoyarse, ahora mismo no podemos ponerla en producción, pero estamos en el camino adecuado para que en un futuro contemos con una buena herramienta de predicción y por tanto de optimización de costes.

En definitiva este trabajo demuestra que el uso de machine learning es fundamental para cualquier empresa que busque ser reactiva y pasar a liderar mediante la anticipación del comportamiento de sus usuarios.